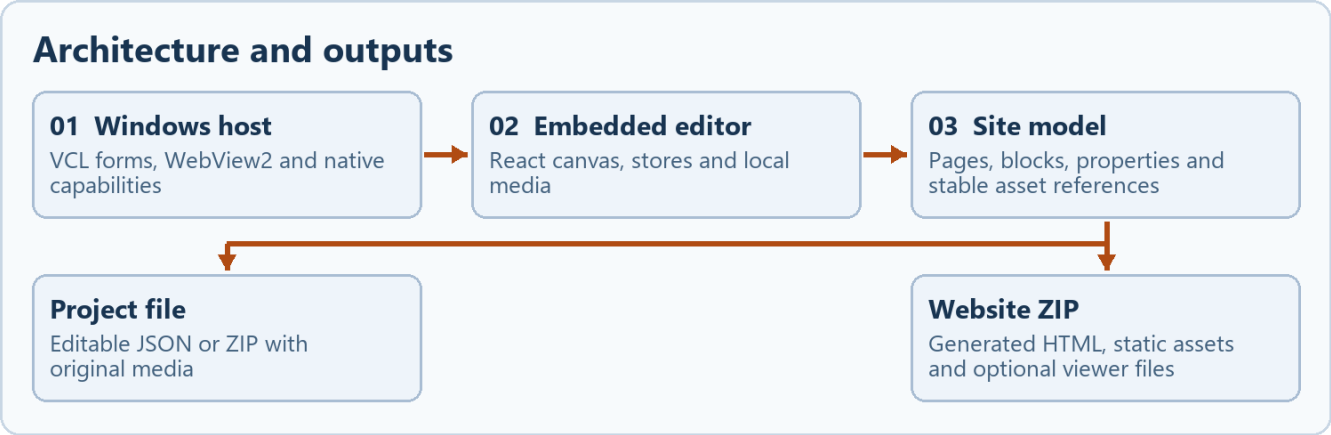
WINDOWS DESKTOP / SOURCE REFERENCE

Delphi Web Studio Pro Engineering Guide

Architecture and implementation

Last changed November 8, 2026 • Source revision 29a650c



The same site model drives the editable file and the independent website export.

For maintainers working on the Windows host, embedded editor or website exporter. Trace where data is owned, how it crosses the native boundary, and what to verify after a change.

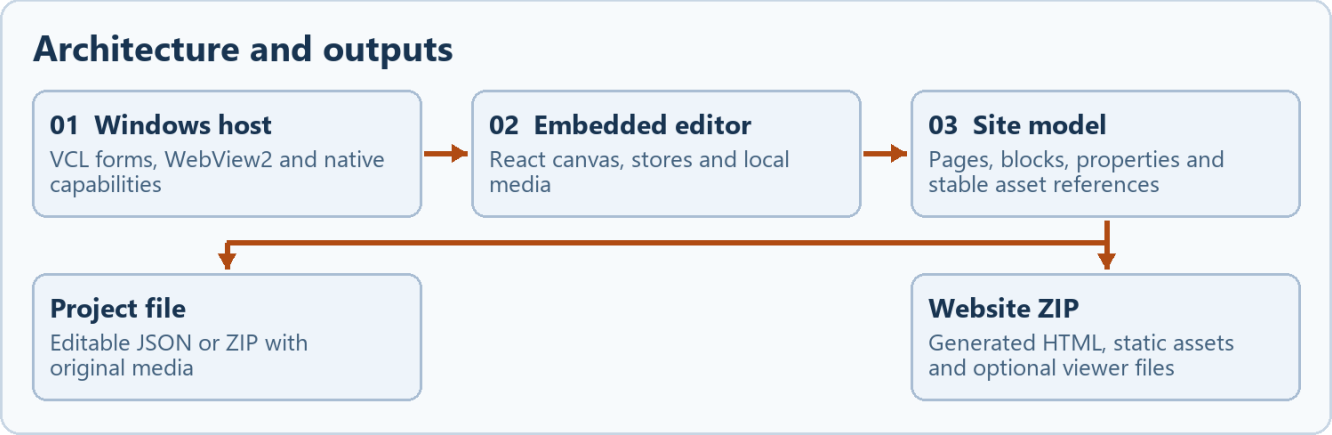
The examples use fictional projects and representative code paths. Paths in source notes are relative to the repository.

Contents

Architecture and boundaries....3
Source tree and ownership....4
Windows startup and WebView2....5
Native bridge and trust boundary....6
State ownership and data flow....7
The versioned project model....8
Mutation history and shared sections....9
Canvas rendering and selection....10
Geometry layering and resize behavior....11
Panels collages and rich text....12
Responsive rendering and theme parity....13
Media storage and asset identity....14
PDF rendering and the bulletin variant....15
Opening a portable project....16
Save handshake and dirty tracking....17
Native file replacement and recovery....18
Static HTML and WordPress import....19
Website export and URL rewriting....20
Generated pages search and visitor features....21
Website review and deployment....22
Local AI process and proposal lifecycle....23
Cloud generation and credential handling....24
Build staging and distribution....25
Tests and regression coverage....26
Debugging by boundary....27
Extending a component safely....28
Source map for maintenance....29

Architecture and boundaries

Delphi Web Studio Pro combines a VCL desktop host with a React editor. The host owns Windows dialogs, privileged file replacement, credential storage, local AI and SFTP. The editor owns the page model, editing state, rendering and generation of portable files. The desktop binary loads the compiled editor from a folder alongside it.



The same site model drives the editable file and the independent website export.

Separate the three outputs

Area	Implementation
Editor bundle	Vite builds index.html and hashed assets. WebView2 loads this bundle as the application UI.
Portable project	The editor serializes its site model and, when media is present, packages original media in a ZIP. This file can be reopened to edit the site.
Website output	The exporter generates HTML and supporting assets for visitors. It consumes the project model but has its own rendering and path rules.

Example. A change to the application menu lives in the .dfm or .pas pair for the host. A change to how a photo resizes lives in the editor. A change to exported page link rewriting lives in the exporter. A change that affects rendered visitor content may need both editor and exporter work.

Source desktop/DelphiWebStudio/DelphiWebStudioPro.dpr · src/App.tsx · src/lib/export-html.ts

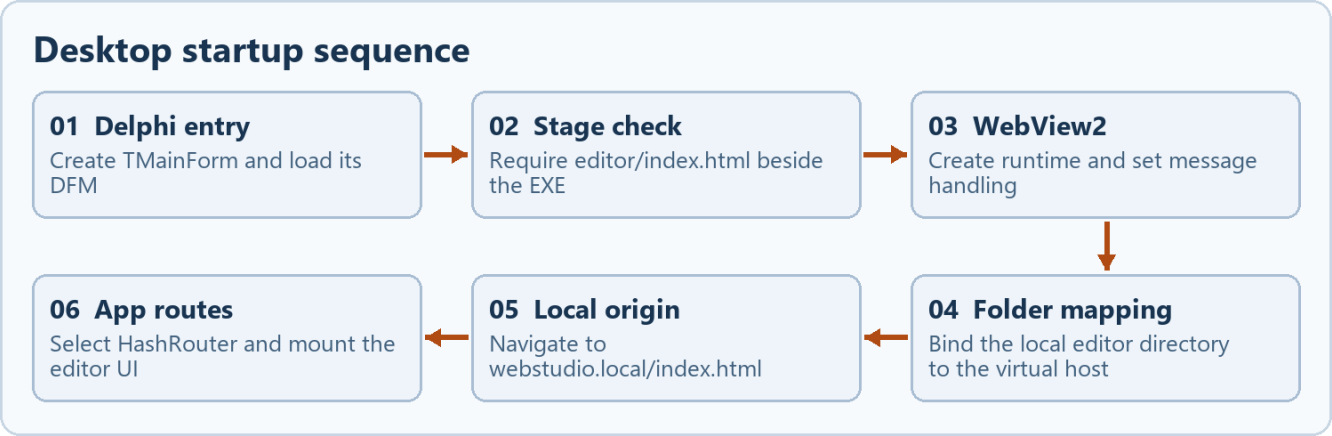
Source tree and ownership

Area	Implementation
desktop/ DelphiWebStudio	IDE-editable .dfm forms, their .pas event handlers, the .dpr entry point, .dproj build settings and resources.
desktop/bridge	Native protocol validation, capability sessions and Windows credential functions.
desktop/project	Native file open, save, backup, restore, SFTP, deployment review, journals and rollback.
desktop/ai	Staging and startup of the local runtime, process ownership and the local HTTP client.
src/routes and src/editor	App routes, editor shell, canvas, toolbar, palette, layer list, property windows and code panels.
src/blocks and src/store	Persisted block types, React renderers and layout wrappers, project and configuration stores, session state.
src/project and src/media	Versioned project parser, portable file commands, importers, checks, asset storage, PDF preview and picker.
src/lib and src/ai	Theme presets, starter templates, content helpers, website renderer, site manifest and AI action validation.
api and tests	Optional server endpoints, browser and unit test suites, native Delphi harnesses and loopback deployment tests.
docs and desktop/installer	Documentation, license material, handoff and Inno Setup packaging inputs.

The package scripts describe the web build. The .dproj and native build setup describe the executable. Find browser behavior through the TypeScript route and store, then trace a Windows-specific operation through the native bridge to its Pascal handler.

Source package.json · vite.config.ts · desktop/DelphiWebStudio/DelphiWebStudioPro.dproj

Windows startup and WebView2



The desktop app starts from local assets and does not require a Vite development server.

DelphiWebStudioPro.dpr initializes VCL, creates TMainForm and enters the message loop. The main .dfm sets wsMaximized, names the controls and connects the event handlers. The paired .pas provides behavior. A TEdgeBrowser fills the client area under the startup status panel.

FormCreate allocates the project path and file identity dictionaries. It checks for editor\index.html beside the running executable before calling CreateWebView. The completion callback checks ICoreWebView2_3, enables messages and maps the local editor folder to https://webstudio.local with cross-origin resource access denied.

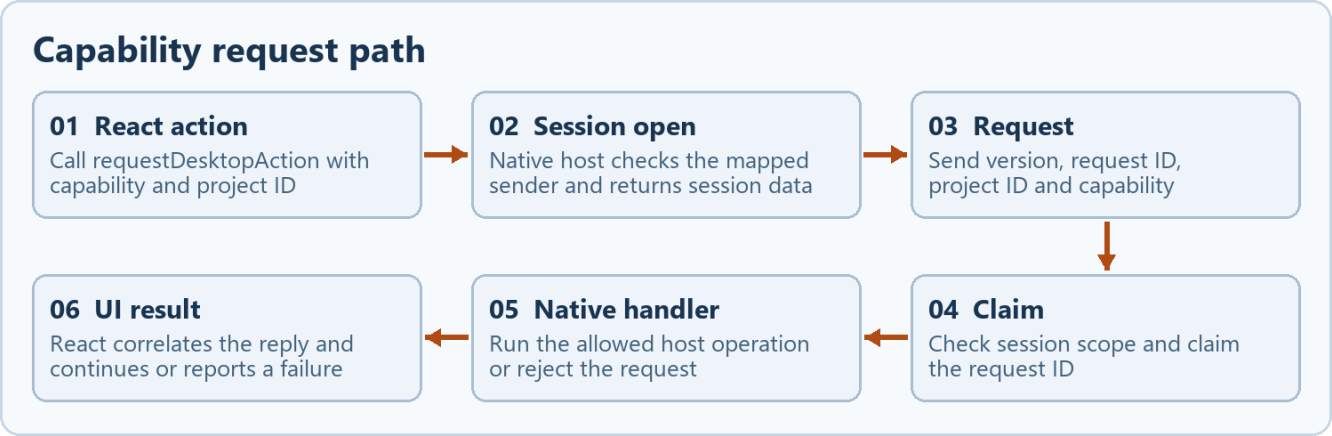
React uses HashRouter for the mapped host, so the browser can navigate to an editor route without requiring a native route server. Standalone browser execution uses BrowserRouter. WebView data is kept under the DelphiWebStudio WebView profile in LocalAppData as configured by the .dfm.

Diagnose a blank startup

Check the native status text and Diagnostics first. Confirm the editor directory has its index and matching hashed assets, the loader architecture matches the executable, and a supported WebView2 Runtime is available. A working Vite development server alone does not establish a valid desktop stage.

Source WebStudio.Main.dfm · WebStudio.Main.pas FormCreate and EditorBrowserCreateWebViewCompleted · src/App.tsx

Native bridge and trust boundary



The privileged operation occurs only after the native handler validates the request.

The React side uses window.chrome.webview when running on the mapped origin. requestDesktopAction opens a versioned capability session for project.open, project.save, media.import or site.export, then sends a request scoped to the current project and capability. In a plain browser the same editor can use the web path without a native session.

WebStudio.Main.pas checks the source before dispatch. WebStudio.Bridge.pas validates the session, capability and request details and claims the request. The host rejects replayed request IDs within a session. The web protocol validator models the contract for tests. Navigation revokes an active native session. Main and frame navigation, new windows and permission handling have their own native event handlers.

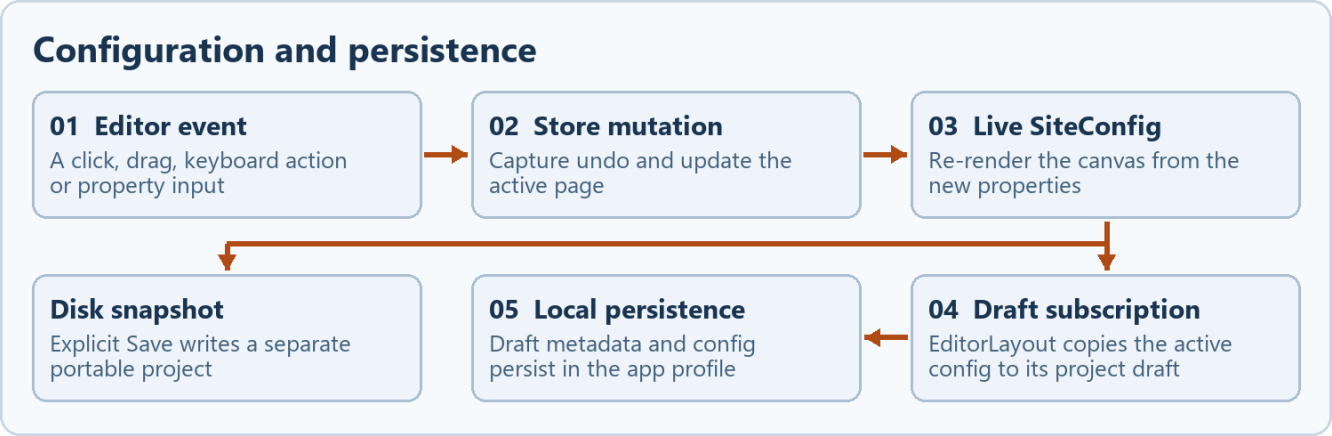
Font and color dialogs, credential messages, dirty-state reports and save-completion messages are additional validated routes in the host. Inspect each handler when changing its fields. Do not use a generic “run command” or arbitrary native file read to extend the bridge.

Trace a request

For a failed project.save capability, first record the kind and request ID without sensitive values. Compare the JavaScript envelope and host source check. Then distinguish a rejected capability from a later rejected save dialog or filesystem operation. The capability reply is separate from write completion. The capability exchange uses distinct request IDs for session.open and the scoped action, and correlates each reply against its own ID.

Source src/desktop/native-bridge.ts · src/desktop/bridge-protocol.ts · desktop/bridge/WebStudio.Bridge.pas · WebStudio.Main.pas

State ownership and data flow



Disk state changes only when a save completes. Local drafting and rendering happen earlier.

Area	Implementation
useConfigStore	The live SiteConfig, active page and undo/redo snapshots. Editing actions update the active page and preserve the Home compatibility block array.
useProjectsStore	Local draft catalog, project metadata, portable-file extension fields, per-project settings and media revision. Persisted in the WebView or browser storage.
useEditorStore	Session UI state, including active project, selection, panel/placement modes, viewport, preview, busy state and native save status.
IndexedDB assets	Original media bytes and metadata are stored per project. The serializable configuration references them by ID or path.
IndexedDB revisions	Explicit snapshots are independent of the editing session and its undo stack.

EditorLayout subscribes to configuration changes and copies active configuration into the relevant draft. A disk save takes a separate snapshot. Editing state and browser drafts can change while the last successfully written project file stays the same.

Source src/editor/EditorLayout.tsx useAutoSaveToProject · src/store/configStore.ts · src/store/projectsStore.ts · src/store/editorStore.ts

The versioned project model

A portable document wraps SiteConfig in SiteProjectFile. The current envelope uses format webstudio and schemaVersion 2. The file projectId can persist across saves while the local draft gets its own identity. Settings and media entries accompany the site model as applicable.

```
{
  "format": "webstudio", "schemaVersion": 2,
  "projectId": "engineering-example",
  "config": {
    "name": "Community Example", "blocks": [],
    "pages": [
      { "id": "home", "name": "Home", "path": "/", "blocks": [] }
    ]
  }
}
```

Each page owns its ordered BlockConfig list. A block has an id, type, variant and property map, plus optional shared-section and visibility metadata. A page can have SEO, article and custom-code data. Partial theme values are resolved against defaults. The top-level blocks array represents Home for compatibility.

validateProjectConfig checks the site structure and paths, requires a Home route for modern multipage data, and protects against invalid JSON values and unsafe content. Unknown extension data can survive the round trip. An unsupported block is a placeholder in editing and can block multipage export.

Source src/blocks/types.ts · src/project/project-file.ts · samples/five-page.webstudio.json

Mutation history and shared sections

Store actions are the transition boundary for page and block changes. mutateActivePageBlocks normalizes pages and resets top-level blocks to Home. updateBlockProps merges properties and propagates through blocks with a matching webstudioSharedSectionId where applicable. Use actions so changes participate in undo and the draft subscription.

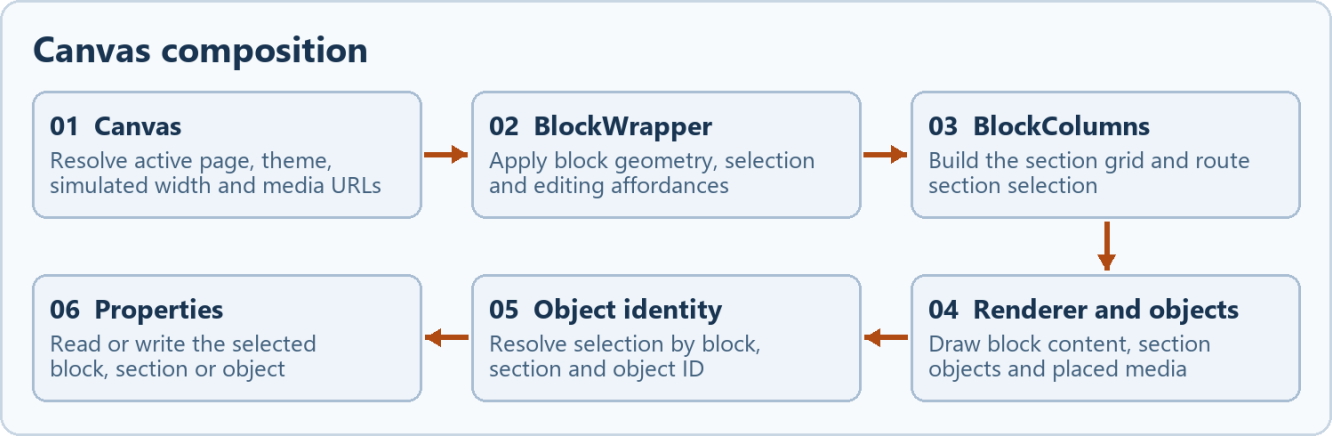
Before a tracked mutation pushUndo takes a structured clone of the config and active page, bounds the undo list at 50, and clears the redo branch. Opening another config resets the session history. Undo snapshots capture the configuration. The media store and native disk writes have their own mechanisms.

Area	Implementation
Shared section	Sharing a Navbar or Footer creates or associates linked copies across pages. Stopping sharing removes the relationship while keeping copies for independent editing.
Reusable section	A project-owned template copy of a block. Inserting it creates an independent block and identity. The saved template can be reused again.
Changing a route	changePagePath validates the new address and can update supported link fields. Export also resolves paths relative to each output file. User-supplied custom code needs separate review.

Test a linked section by editing one copy, reading the other copies, undoing and checking both Home and a secondary page. Testing only the active block misses cross-page effects.

Source src/store/configStore.ts · tests/page-history.test.ts · src/blocks/BlockWrapper.tsx

Canvas rendering and selection



Selection identity connects the visible object to the relevant settings fields.

Canvas selects the active page, builds temporary media URLs and sets theme CSS variables on the width-constrained viewport. BlockWrapper supplies editing selection, dimensions, drag behavior and context actions around the registered renderer. BlockColumns provides section containers. SectionObjects and PlacedMedia support independent objects and older placement data.

Canvas pointer capture coordinates placement, selection and group interactions. SectionObjects records a selected object as block ID, section number and object ID. Layers and context menus resolve objects by identity. RightSidebar hosts floating settings and routes to the focused section, object, media entry or composite item.

Single clicks select objects. Double clicks enter editable text where supported. Keyboard nudges, handles and move cursors apply at the object level. Editor Preview removes editing chrome and renders the same configured blocks with editing disabled. It follows supported internal page and anchor links.

Source src/editor/Canvas.tsx · src/blocks/BlockWrapper.tsx · src/blocks/registry.tsx · src/editor/RightSidebar.tsx

Geometry layering and resize behavior

The same page can contain whole blocks, legacy placed media and newer objects inside sections. Their identifiers and coordinate systems differ. Block width and left position are percentages of page width. Section object x and width are percentages of the section, while y and height use pixels. Higher local layer values draw in front.

BlockColumns clamps the number of visible sections to one through four. It uses a grid and persists divider changes as section widths. Hidden extra sections can retain stored contents. SectionObjects computes a prospective change from pointer movement and the container width. The move cursor and resize handles belong to the current selected object, independent of its content.

```
// Example geometry inside a section that is 600 px wide
{ "id": "park-image", "type": "image", "x": 10,
  "y": 24, "width": 40, "height": 135, "layer": 2 }
```

In the example, the object begins 60 px from the section's left edge and spans 240 px. A drag into a differently sized section recalculates its horizontal coordinates using the destination bounds. moveSectionObject moves the object, its attached children and rich-text properties as one configuration change.

Image movement preserves the grab offset. Aspect-ratio resize uses measured image geometry where applicable. Blue guides compare nearby edges and centers. Snapping applies the calculated adjustment when enabled. The selected-object toolbar requires compatible same-section items for alignment or distribution.

Source src/blocks/BlockColumns.tsx · src/blocks/SectionObjects.tsx · src/blocks/BlockWrapper.tsx · src/editor/SelectionToolbar.tsx

Panels collages and rich text

A section panel can own objects by parentId. Moving it also moves attached children. Overlap alone is a geometric relationship, while parentId establishes ownership. Layer selection and context menus give access to a panel even when a child covers its surface.

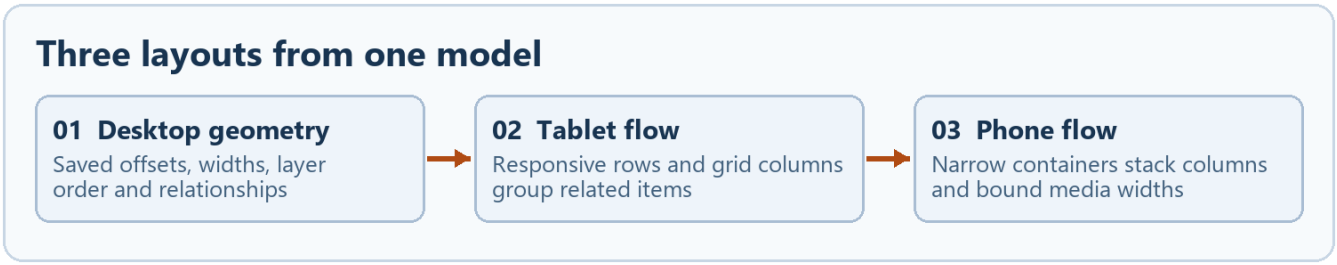
A collage can be a standalone block or a section object that contains its own images. moveCollageBlockToSection creates a destination object and removes the source block in one undoable mutation. It rejects a source with shared or additional section content to avoid dropping unrelated data. Individual collage images keep their own geometry and layer order.

RichText binds an editable field to its stored text and formatting. The browser's selection identifies the range used for font, color, emphasis and alignment operations. Rich runs are persisted alongside plain-text data. Section object formatting is keyed by object ID in sectionObject_<id> and sectionObject_<id>Rich properties. Code that duplicates, transfers or deletes a text object must account for those values.

Save dispatches webstudio-flush-text before taking its snapshot and also blurs a focused contenteditable. The dirty handshake tracks an input version to detect new edits during save. Avoid remounting or replacing editable DOM during a live selection without preserving or intentionally ending the edit.

Source src/blocks/RichText.tsx · src/blocks/SectionObjects.tsx · src/blocks/collage/CollageBlock.tsx · src/store/configStore.ts

Responsive rendering and theme parity



Both CSS surfaces derive responsive display from the same saved geometry.

The desktop arrangement can use absolute object positions. At narrower container widths, responsive rows and columns can replace absolute positions with flow. The rules in src/index.css and src/lib/export.css must stay aligned. They reflow at container widths of 940 and 700 px. Separate device-visibility checks have their own ranges.

SectionObjects groups related items from the saved desktop arrangement and provides grid columns, spans and width hints as CSS variables. Tablet-sized layouts can keep related groups side by side. At phone widths columns stack, images remain bounded and attached panel content flows together. Canvas presets simulate a container width, which can be narrower than the app window.

Example of a parity fix

The page backdrop is theme bg0. Canvas.tsx now uses --color-bg-0 for exposed gutters. The HTML exporter sets body background-color from the same theme value. The block's own background remains controlled by its block properties. A regression test should use an inset Navbar and a contrasting Hero on a white page, then inspect Editor, Preview and exported output.

The component wrapper also applies a custom block background through resized space. The canvas, block and child section are different background surfaces. Inspect their computed styles individually when diagnosing an unexpected strip.

Source src/index.css · src/lib/export.css · src/editor/Canvas.tsx · src/lib/theme-presets.ts · src/lib/export-html.ts

Media storage and asset identity

ProjectMediaLibrary opens IndexedDB webstudio-media at version 3 and uses an assetsV2 store keyed by project ID and asset ID. The stored record keeps a Blob, SHA-256, MIME type, size and source metadata. The library supports reuse, separate copies or skipping when file hashes match. Folder and filename metadata support searching and paging in the picker.

The editor imports and validates media before referencing it in a block. Where applicable it extracts dimensions or a thumbnail. Canvas maps IDs and stable paths to temporary object URLs for rendering and revokes those URLs when the effect is replaced or unmounted. The serialized file carries the persistent paths. Blob URLs belong to a running browser session and must not become portable links.

Area	Implementation
Rename	Changes media metadata without requiring a new identity. Undo renames restores recorded prior names.
Replace	Substitutes bytes for an existing asset and can preserve the previous version for undo. The project media revision changes so the canvas refreshes the rendered media.
Copy and reopen	Copies or restores assets into the destination project namespace so two drafts can hold separate media.

Example. A page object references assets/<id>-park.png. Saving validates its bytes and checksum. Preview resolves that path to a Blob URL. Export keeps the stable path and adjusts the relative link when writing nested HTML.

Source src/media/media-library.ts · src/media/MediaPanel.tsx · src/media/MediaLibraryDialog.tsx · src/editor/Canvas.tsx

PDF rendering and the bulletin variant

PdfBlock uses PDF.js and its worker to render the current page to a canvas. Source, page, zoom and dimensions determine the active render. Page selection and search state are scoped to the document. The expanded bulletin dialog keeps the toolbar separate from the scrollable PDF view. Page-fit, zoom, search and the original PDF link are distinct actions.

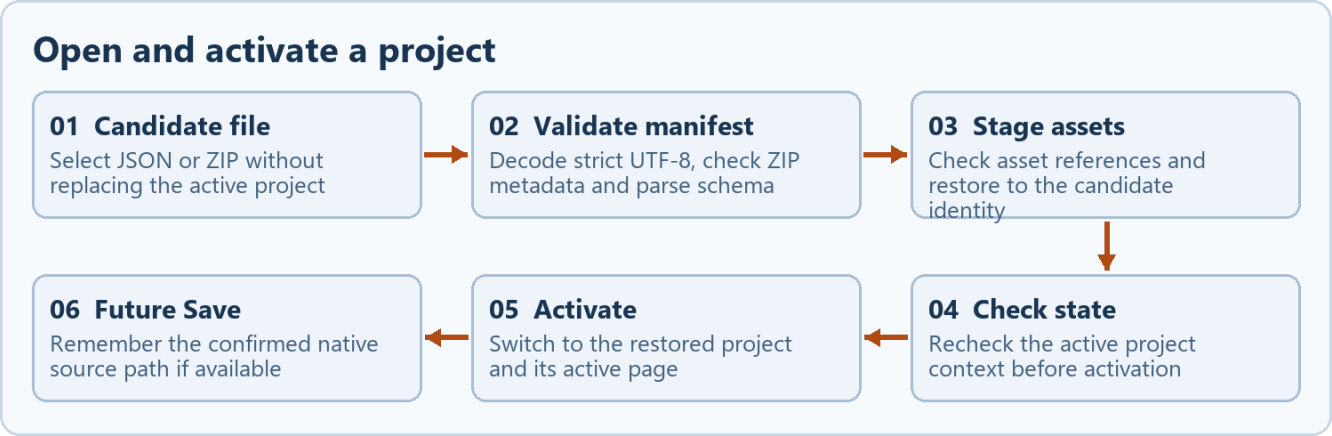
DocumentLibraryBlock treats variant weekly-bulletins as a list of up to five valid PDF entries. The first is current. The picker prepends the selected asset and avoids retaining a duplicate row for the same asset. In the editor the path resolves to a Blob URL and a click opens the dialog. Exported bulletin links target the original PDF.

A standalone PDF viewer block differs from a bulletin list. The website exporter can package PDF.js and viewer assets for supported embedded PDF output. A general document library can expose a search box and document download links. Preserve these distinctions when changing a generic media action.

Test with a fictional two-page PDF. Confirm page 2, search, Fit page, scroll and original bytes. Then export and confirm the visitor link points to the correct relative PDF path.

Source src/blocks/pdf/PdfBlock.tsx · src/blocks/media/DocumentLibraryBlock.tsx · src/media/viewer.html · src/lib/export-html.ts

Opening a portable project



The candidate becomes active only after the project and its media are ready.

ProjectFileCommands reads JSON directly or finds the versioned manifest in a ZIP. readProjectZipManifest checks bounds and recorded CRC before decoding strict UTF-8. parseProjectFile validates or migrates a supported older configuration. Media paths, sizes and digests are checked as assets are restored.

The candidate receives a local draft identity. Asset restoration happens against that staged identity before the project becomes the active editor session. If restoration fails, tracked staged assets are removed and the existing project remains the working copy. Some steps also check that the active project has not changed during the operation.

The native host records the opened file's path and identity for later Save. A portable document can retain its original projectId when the local draft ID changes. Compare these identities separately when tracing a reopened or duplicated project.

Example failure

Change the bytes of an image in a synthetic ZIP without updating its manifest. Opening must report the mismatch and keep the current project. It must not silently substitute a broken asset or set the candidate as the active project.

Source src/project/ProjectFileCommands.tsx · src/project/project-file.ts · desktop/project/WebStudio.ProjectOpen.pas

Save handshake and dirty tracking



A completed transfer is not enough to clear a newer edit.

ProjectFileCommands flushes editable text, records the active project/configuration, validates references, and creates a JSON or ZIP snapshot. Settings and asset metadata go into the manifest. If media is present, the ZIP includes project.webstudio.json and the original asset bytes. The deployment access key is removed from portable settings.

In the desktop build the snapshot download is tagged with a token. Delphi intercepts the download and selects the current path or a new destination. The renderer records the project ID, saved signature and input version while waiting for confirmation. A generic browser download can be requested without proving that a Windows file was committed.

The host reports project.save.complete with the token and path after commit. The editor compares the completed snapshot with the current project and sends project.save.confirmed with its clean flag. Dirty state includes pending or newer text, config, name, settings and media changes. A late acknowledgement must not mark later edits as saved.

FormCloseQuery gates close when a save is underway or the editor is dirty. A save-on-close request waits for a matching successful clean confirmation. Cancelled or failed saves leave the app available to recover the work.

Source src/project/ProjectFileCommands.tsx · desktop/DelphiWebStudio/WebStudio.Main.pas FormCloseQuery and TDownloadStateHandler.Invoke

Native file replacement and recovery

Native save separates the final destination from the staged download. The completion handler validates the staged file and records destination identity. New files use a write-through move after flushing. Replacement uses Windows ReplaceFile and checks identity to reduce the chance of replacing a changed destination. The save path reports failure if expected data or identity checks fail.

Replacement has a temporary recovery policy for exceptional cases. Routine successful saves clean up temporary recovery material. A staged file can be offered for recovery after interruption. A user-requested backup ZIP is a separate operation. Keep it until the restored project has been reopened and checked.

Area	Implementation
Explicit project backup	SaveProjectRecoveryZip writes a package from the current confirmed file. It works with disk state, which can be older than an unsaved draft.
Restore backup	RestoreProjectRecoveryZip writes and verifies the recovery contents before completing a restore. Open the result to verify the model and media.
Local revision	ProjectRevisions stores a separate Blob and checksum in IndexedDB. Opening creates another draft. A local revision is not the native backup ZIP.

Tests for replacement should include an existing target, an external file change, cancellation, a new filename, interrupted transfer and saving during later text changes. Use disposable paths. These cases exercise different branches.

Source desktop/project/WebStudio.ProjectSave.pas · WebStudio.Main.pas · src/project/project-revisions.ts · tests/native/ProjectReplacementSmoke.dpr

Static HTML and WordPress import

The static importer accepts selected directory files and asset candidates. It maps supported pages and references into a project, reports skipped files or unresolved references and retains source snapshots where supported. ProjectFileCommands shows the review before creating the draft and stages approved local images in the media library.

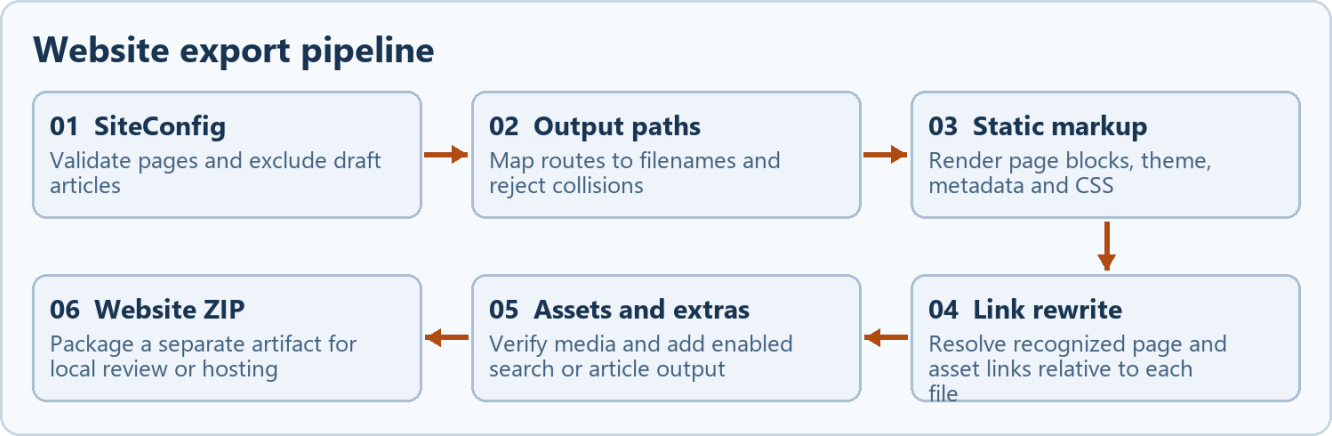
The WordPress importer accepts exported XML and converts supported pages and posts into editable blocks and article metadata. The review surfaces counts and conversion issues. A WordPress theme, plugin or live service is outside the imported model.

Imported content and explicitly entered custom code have different trust paths. The editor can inspect source from JsonDrawer and preview custom HTML/CSS in a controlled view. The website export deliberately writes configured custom CSS and JavaScript and inserts custom HTML. Treat it as executable content at the hosted destination.

Test a folder with Home, a nested page, one local image and a broken reference. Verify its review, the new draft and the exported relative links. For XML, check an unpublished post and malformed input as separate cases.

Source src/project/static-html-import.ts · src/project/wordpress-import.ts · src/project/ProjectFileCommands.tsx · src/editor/JsonDrawer.tsx

Website export and URL rewriting



The generated site has its own files and does not contain the editable desktop project.

exportSiteZip validates and normalizes the model, omits draft articles, and refuses unsupported blocks in exported pages. Home becomes index.html. A route such as /events/cleanup becomes events/cleanup.html. The exporter rejects case-insensitive collisions, incompatible file and folder paths, and conflicts with generated assets.

For each page it passes that page's blocks to exportSiteToHTML. The renderer uses dedicated HTML templates and React static markup for selected components. It also applies page SEO overrides, the resolved theme and export CSS. The multipage pass rewrites known internal routes and asset paths relative to the output file while preserving query and anchor suffixes.

```
Project route      /events/cleanup
Output file        events/cleanup.html
Link to Home       ../index.html
Local image        ../assets/<id>-park.png
```

It checks referenced original media against size and SHA-256 before packaging. ZIP timestamps are normalized. The editor project's ZIP is a different artifact and may contain media that no exported page references.

The export does not recreate the editor or rely on its React runtime for site editing. The resulting site can still use configured external embeds, fonts, analytics or form services. Test the output in the deployment context.

Source src/lib/export-html.ts exportSiteToHTML and exportSiteZip · src/lib/export.css · tests/multipage-export.test.ts

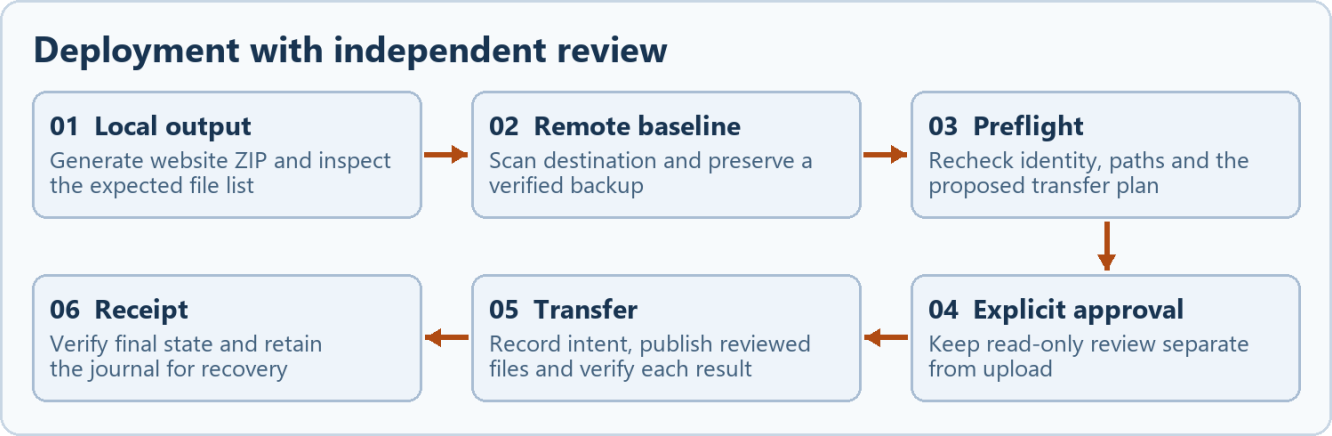
Generated pages search and visitor features

Area	Implementation
Search	Opt-in site search generates its page and static search data from exported content. It is separate from editor-side Check site. Review the generated output after changing search indexing or exclusions.
Articles	Draft articles stay in the project. Ready articles export and can appear in the generated index. Author, summary and tags travel with the page metadata. A valid public base URL supports canonical links, sitemap and the article RSS feed.
Navigation	An ordinary page's links come from its configured Navbar or other content. The generated search and article index can reuse the Home Navbar. Sharing an editable Navbar is separate from this export behavior.
Contact	The project webmaster email supplies the default recipient when a Contact block has none. Preview shows behavior without transmitting. Export can use a valid configured HTTPS endpoint or open the visitor's email application.
Anchors	Names become element IDs when valid. Link handling distinguishes external URLs, page routes, same-page hashes and media.

Example. Add a ready /news/cleanup article with a local photo and an About link. Verify the photo and relative About link in that nested HTML file, then verify the article index and search include the article. Change the article to Draft and test the new output and any inbound links.

Source src/routes/Settings.tsx · src/project/site-checks.ts · src/blocks/contact/ContactBlock.tsx · src/lib/export-html.ts

Website review and deployment



An interrupted deployment is investigated against its journal and fresh destination state.

The browser review generates a local Website ZIP, lists paths, and can calculate or compare a website manifest. Review and export can run without uploading. createDeploymentPlan compares a proposed ZIP with a destination scan and backup. Native SFTP is separately reviewed in the desktop forms.

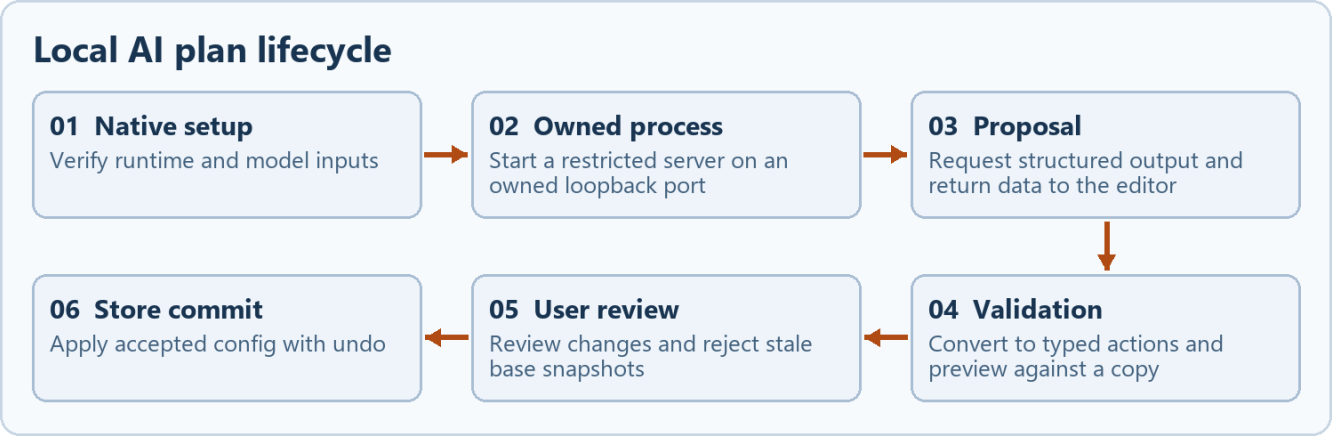
The host uses Windows OpenSSH and configured key and trusted-host files. It can test the connection, read a snapshot, scan the destination and back up supported files. Before upload the deployment units revalidate inputs and rescan or recheck expected remote state. An explicit publish action is separate from the read-only operations.

The executor records transfer intent before mutation and verifies completed work in a journal. It distinguishes new and replacement files. Replacement has private recovery requirements. Rollback has a separate review and receipt. Added-file rollback can retain those files in private recovery instead of treating the operation as a blind deletion.

The portable-project build keeps the optional hosted API path distinct from the desktop SFTP flow. GitHub stores the software source. Saving a project or building an export does not deploy to GitHub Pages.

Source src/lib/website-manifest.ts · desktop/project/WebStudio.SftpInventory.pas · WebStudio.DeploymentPreflight.pas · WebStudio.DeploymentExecutor.pas · WebStudio.DeploymentJournal.pas · WebStudio.RollbackExecutor.pas

Local AI process and proposal lifecycle



The model proposes changes. The validated editor action and user approval commit them.

The Local AI VCL form collects a supported runtime ZIP, GGUF model and request. The native host verifies the supported runtime and model, then starts a restricted llama.cpp server on an allocated loopback port. The process uses an API-key file and a Windows job object. The host checks port ownership and waits for readiness before sending the chat request. Inference runs off the UI thread.

The returned plan is data. React converts a supported proposal to typed actions, runs those actions against a copy, validates the result and presents a review. The five-page planner requires a blank single-page project. Single-page and color requests can be scoped to an existing project.

acceptSiteActions compares the active configuration against the preview base snapshot. If it changed, approval is rejected and the plan must be reviewed again. An accepted proposal is committed through the configuration store with undo. Native process teardown is separate from accepting the plan.

Example. A proposal to add /volunteer can produce createPage and addBlock actions. If a user adds /volunteer while review is open, the stale proposal must not overwrite the new page.

Source WebStudio.LocalAIForm.pas · desktop/ai/WebStudio.LocalAIHost.pas · WebStudio.LocalAIProcess.pas · WebStudio.LocalAIClient.pas · src/ai/site-actions.ts · src/ai/LocalPlanReview.tsx

Cloud generation and credential handling

The optional browser generation path in generate-site.ts can use a Gemini key directly, fall back to the hosted generation endpoint where available, and fall back to labeled starter content. Cloud generation is off by default and its enablement is session-scoped. The native local AI path is separate. Drafting and project export can operate without a cloud AI key.

Desktop application settings can persist a Gemini key through Windows Credential Manager for the current Windows user. The renderer requests or updates the credential through the native bridge. The credential is not part of the project model. Hosting credentials have a separate purpose and are excluded from the portable project save.

Changes involving secrets should preserve the origin boundary and avoid diagnostic output containing values. Test blank keys, cancellation, a rejected origin and a valid round trip without retaining test credentials. When examining API behavior, distinguish native local inference, a direct remote call and the hosted endpoint.

Source src/routes/Settings.tsx · src/lib/generate-site.ts · api/generate.ts · desktop/bridge/WebStudio.Credentials.pas · src/desktop/native-bridge.ts

Build staging and distribution



Win32 and Win64 each need a coherent stage, even when both share the same editor assets.

The editor uses TypeScript, React, Vite and Tailwind. npm run build invokes tsc -b then vite build. The dependency lockfile is the reproducible starting point for npm ci. Desktop expects the built dist files staged as editor\ next to the executable. Node.js is needed for development, not for running that installed bundle.

Open desktop/DelphiWebStudio/DelphiWebStudioPro.dproj in RAD Studio for the VCL project. Use the configured 37.0 toolchain and build Win32 and Win64 deliberately. Keep generated units in dcu and stage the final executables under bin\Win32 and bin\Win64. Match WebView2Loader.dll to the binary architecture.

```
bin\Win64\
  DelphiWebStudioPro.exe
  WebView2Loader.dll
  editor\index.html
  editor\assets\...
```

A change confined to the embedded UI requires rebuilding and staging the editor bundle. The host executable changes when Delphi code, form resources or native build inputs change. Verify the packaged index references assets that exist in both stages. An old executable timestamp alone cannot identify the embedded editor revision.

The Inno Setup script consumes a verified x64 stage and checks the separate WebView2 Runtime prerequisite. The repository includes installer and license material. Treat clean-Windows acceptance and release validation as separate from staging a local build.

Source package.json · vite.config.ts · DelphiWebStudioPro.dproj · desktop/installer/DelphiWebStudioPro.iss · desktop/installer/README.md

Tests and regression coverage

Area	Implementation
TypeScript checks	Run npm run lint, npm test and npm run build. These exercise source and bundle checks. GitHub's checks workflow is manually triggered in the inspected revision.
Project and media tests	project-file.test.ts, project-storage.test.ts, project-revisions.test.ts and media-library.test.ts cover model, storage and archive behaviors.
Editor tests	editor-selection.test.ts, floating-settings.test.ts, selected-object-properties.test.ts, navbar-properties.test.ts and block-columns.test.ts focus on state and markup contracts.
Export and AI tests	export-html.test.ts, multipage-export.test.ts, website-manifest.test.ts and site-actions.test.ts exercise output, path rules and proposal acceptance.
Browser test	tests/browser/professional-ui.mjs targets the editor UI. Use isolated profiles and a fixture so edits and browser storage are disposable.
Native tests	tests/native contains Delphi harnesses for bridge, save, recovery, export and AI. SFTP scripts support controlled loopback testing. Run native coverage against the relevant loader and Win32 or Win64 target.

A passing unit suite does not establish that a deployed site or a native dialog works. Test the actual affected path. For a save fix, inspect disk bytes and reopen the result. For responsive layout, check Editor, Preview and the extracted HTML at the same widths.

Source .github/workflows · tests/browser/professional-ui.mjs · tests/native · package.json

Debugging by boundary

Area	Implementation
Blank app window	Start at FormCreate, the native startup status and Diagnostics. Confirm the mapped editor directory, bundle hash names and matching runtime/loader.
Cannot select or move	Inspect the hit target, active editing state, block ID, section and object ID. Compare Canvas capture, SectionObjects and Layers before changing store logic.
Change appears in editor only	Find the mutation and saved properties. Trace the corresponding static renderer and CSS. Check whether a legacy placement path or custom code owns the final output.
ZIP opens without media	Verify manifest paths, asset sizes and digests, then inspect the project's IndexedDB namespace. Check for persistent references that accidentally used object URLs.
Save remains dirty	Trace webstudio-flush-text, the saved signature, input version and token. Compare project.save.complete and project.save.confirmed. Inspect the native save error before touching user files.
Wrong file on the host	Compare the local export, reviewed manifest, destination scan and deployment receipt. Recheck the destination rather than rerunning an interrupted upload blindly.

Keep diagnostic test data separate from user projects. Use sample routes and media. Record the source revision, stage architecture, exact trigger and affected boundary in any handoff.

Source WebStudio.Main.pas · src/editor/Canvas.tsx · src/project/ProjectFileCommands.tsx · src/lib/export-html.ts

Extending a component safely

Start with the persisted property contract. Determine what existing project files supply and whether the new property needs a default or migration. Keep a stable object identity. Use a store action so the edit preserves undo, project drafts and linked copies.

Wire the component across the places that own it. The block metadata provides palette defaults, the registry selects the React renderer, and PropertiesPanel exposes fields. BlockWrapper and SectionObjects provide the relevant manipulation affordances. Export needs matching rendering, validation and responsive CSS for visitor behavior.

Worked example for an optional appearance field

Define what “unset” means and render an older project without the new field. Set it through Properties and confirm undo. Export both versions, inspect the markup and styles, then reopen the saved project. Check desktop and narrow containers, as a desktop-only geometry assertion can miss the responsive path.

A format change should exercise import, save, reopen and export. A native change should exercise the host and bridge. Document which branch was verified, and remove test profiles, staging files and harness outputs after the work.

Source src/lib/block-metadata.ts · src/blocks/registry.tsx · src/editor/PropertiesPanel.tsx · src/store/configStore.ts · src/lib/export-html.ts

Source map for maintenance

Area	Implementation
Startup and forms	desktop/DelphiWebStudio/DelphiWebStudioPro.dpr · WebStudio.Main.dfm · WebStudio.Main.pas
Native message contract	src/desktop/native-bridge.ts · src/desktop/bridge-protocol.ts · desktop/bridge/WebStudio.Bridge.pas
Page model and undo	src/blocks/types.ts · src/store/configStore.ts · src/store/projectsStore.ts · src/store/editorStore.ts
Canvas and layout	src/editor/Canvas.tsx · src/blocks/BlockWrapper.tsx · src/blocks/BlockColumns.tsx · src/blocks/SectionObjects.tsx · src/index.css
Project and assets	src/project/ProjectFileCommands.tsx · src/project/project-file.ts · src/project/project-revisions.ts · src/media/media-library.ts
Native storage	desktop/project/WebStudio.ProjectOpen.pas · WebStudio.ProjectSave.pas
Export and SFTP	src/lib/export-html.ts · src/lib/export.css · src/lib/website-manifest.ts · desktop/project/WebStudio.SftpInventory.pas · WebStudio.DeploymentExecutor.pas
Local AI and proposals	desktop/ai/WebStudio.LocalAIHost.pas · WebStudio.LocalAIProcess.pas · WebStudio.LocalAIClient.pas · src/ai/site-actions.ts
Packaging and checks	desktop/installer/DelphiWebStudioPro.iss · .github/workflows · package.json · tests/native

This guide describes the code at revision 29a650c. Earlier milestone notes in docs are useful historical records. Check their dates before using them as a description of the current implementation.